

構造体の代入 — レジスタに載らないものを運ぶ

今日のゴール

`q = p;` のように、構造体をまるごとコピーできるようにする。ネストした構造体（Line 中の `Point`）やポインタ経由（`*dst = *src`）も動くようにする。

1 この回の位置づけ

- 言語機能発展シリーズ（選択制）。前提はコマ 12（構造体）
- コマ 12 の資料が「構造体のネスト、構造体代入、グローバル構造体変数は扱わない」と明示的に先送りした宿題の回収
- `mycc.py` は書き換ええない。ラッパー `langcc.py`（完成品）が `Assign` の生成だけ差し替える
- 編集するのは `structcopy.py` の 2 つの関数

注意

引数の値渡し（`f(p)` で構造体を渡す）と戻り値の値渡しは、この回では扱わない。どちらも RV64 の呼び出し規約に踏み込む必要があり（16 バイト以下はレジスタ 2 本、それより大きければ呼び出し側が領域を用意して隠し引数で渡す）、分量が 2 倍以上になるためである。発展課題で設計だけ扱う。いままで通り、関数へはポインタで渡す（`f(&p)`）。

2 いまはどうなっているか

```
Point p; Point q;  
p.x = 3; p.y = 4;  
q = p;  
return q.x + q.y;    /* 7 のはず */
```

いまのコンパイラは、これをエラーにせず、間違った答えを返す。

ネストしたメンバへのアクセス（`l.a.y`）は、実は既に正しく動く。コマ 12 が「ネストは扱

わない」としていたのは慎重を期しただけで、`_struct_defs` は入れ子のオフセットをちゃんと持っている。壊れているのは代入だけである。

3 なぜ壊れるのか — レジスタに載らない

いままでの代入は、こういう形だった。

```
左辺のアドレスを求める → 退避
右辺の値を a0 に載せる
a0 を左辺のアドレスへ書き込む
```

「右辺の値を a0 に載せる」——ここが構造体では成り立たない。`Point` は 8 バイト、`Line` は 16 バイト、もっと大きい構造体もある。レジスタ 1 本に載らないものは、この方式では運べない。

いまの実装は、構造体でも同じ道を通して `ld / sd` で 8 バイトだけ写す。`Point` (ちょうど 8 バイト) なら偶然動きそうに見えるが、型サイズの計算がずれて `lw` (4 バイト) になり、`y` が写らない。結果として `q.y` はゴミになる。

4 どう直すか — アドレスからアドレスへ

構造体の代入は、値の移動ではなくメモリのコピーである。

```
左辺の「アドレス」を求める → a1
右辺の「アドレス」を求める → a0      ← 値ではない
a0 から a1 へ、size バイト写す
```

右辺に `codegen()` ではなく `codegen_lval()` を使うのが要点である。コマ 9 で学んだ「`rvalue` と `lvalue`」の区別が、ここで効いてくる。構造体の代入では、右辺も `lvalue` として扱う。

コピーは大きい単位から行う。

```
ld a2, 0(a0)      ;; 8 バイトずつ
sd a2, 0(a1)
ld a2, 8(a0)
```

```
sd a2, 8(a1)
lw a2, 16(a0)    ;; 残りが4バイト以上なら4バイト
sw a2, 16(a1)
lb a2, 20(a0)    ;; 最後の端数は1バイト
sb a2, 20(a1)
```

補足

これは **memcpy** である。いま書いたループは、libc の `memcpy(dst, src, n)` がやっていることそのものである。実際のコンパイラも、構造体の代入を `memcpy` の呼び出しに変換することが多い（サイズが小さいときは、いまのように命令を並べて展開する）。「構造体の代入」という言語機能の正体は、**メモリのコピー**である。

5 代入式の値は何か

Cでは代入も式なので、`a = (b = c)` のように値を持つ。構造体の代入の場合、その値は「代入された構造体」である。

レジスタには載らないので、この回では**左辺のアドレス**を `a0` に残しておく。`p = q = r;` のような連鎖でも、次の代入が `codegen_lval` でアドレスを求め直すので辻褄が合う。

6 実装

Step	実装対象	内容
1	<code>is_struct_assign</code>	左辺の型が構造体かを判定する
2	<code>gen_struct_copy</code>	両辺のアドレスを用意して <code>size</code> バイト写す

```
python3 langcc.py tests/assign.c | grep -A6 "ld a2"
python3 langcc.py tests/nested.c | grep -c "sd a2"    # 16 バイト = 2 回
```

7 テスト

```
python3 check.py      # 判定と生成命令列を確認（未実装は SKIP）
python3 golden.py     # 構造体代入のテスト + fixed15
```

テスト	確認内容
<code>assign.c</code>	<code>q = p</code> でコピーされ、その後 <code>p</code> を変えても <code>q</code> が変わらない
<code>nested.c</code>	<code>l.a = p</code> （ネストしたメンバ）と <code>m = l</code> （16 バイト）
<code>via_ptr.c</code>	<code>*dst = *src</code> （ポインタ経由）

`assign.c` の「コピー後に元を変えても影響しない」が本質的である。参照ではなく複製であることを、そこで確かめている。

8 発展課題

1. 引数の値渡し: `int area(Point p)` を書けるようにする。RV64 の規約では、16 バイト以下の構造体はレジスタ 2 本に分けて渡し、それより大きければ呼び出し側がコピーを作ってアドレスを渡す。簡易版として「常にコピーを作ってアドレスを渡す」方式を設計してみる（libc とやり取りしないなら、それでも動く）
2. 戻り値の値渡し: `Point make(int x, int y)` は、呼び出し側が領域を用意し、そのアドレスを隠し引数として渡す方式で実現できる（sret と呼ばれる）。なぜ「戻り値を `a0` に載せる」では足りないのか
3. 大きな構造体: 1KB の構造体を代入すると、命令が何百行も並ぶ。サイズが一定を超えたら `memcpy` を呼ぶように変える
4. グローバル構造体変数: `Point origin;` をグローバルに置けるようにする（`.bss` のサイズ計算は `_struct_defs` から引ける）
5. 構造体の比較: C では `p == q` は書けない。なぜ代入はできて比較はできないのか

補足

コラム:「値」の大きさが違うということ。int の代入も構造体の代入も、C の文法上は同じ `=` である。しかし機械語のレベルでは、片方は 1 命令、もう片方はコピーのループになる。言語が同じに見せているものが、実装では違う——これはコンパイラを書いて初めて実感できることの 1 つである。配列が代入できないのは、この延長で「配列は式として値にならない」と決められているからで、逆に構造体は「値になる」と決めた結果、コンパイラがコピーを生成する義務を負っている。